Predicting CVSS and CWE from CVE Descriptions with Fine-Tuned Small Language Models

Tara Dixit **Jessie Dong** **Summer Li**

Stanford University

{taradixit, jessiedong, summerli}@stanford.edu

Abstract

Security teams triage newly disclosed vulnerabilities by their CVSS vector, the severity tier it implies, and their CWE weakness class. Until April 2026 the National Vulnerability Database (NVD) supplied these labels for nearly every CVE. NIST has since limited enrichment to high-priority vulnerabilities, and most new CVEs now arrive as a text description with none of the fields that triage depends on. We formalize the problem of recovering these fields as *CVE enrichment*: given only the description, predict the eight CVSS v3.1 components, the base score, the severity tier, and the CWE. Progress on this task requires evaluation that respects time, because vulnerabilities in the same product recur across years, so we train on CVEs from 2022 and 2023 in a pinned NVD snapshot and test on all 36,325 usable CVEs from 2024. We compare TF-IDF classifiers, the 3.8B-parameter Phi-3.5-mini used zero-shot, and the same model fine-tuned with LoRA, and we evaluate each under two scoring schemes that either predict the CVSS score directly or compute it from the predicted vector with the official formula. The fine-tuned model is the strongest system on every field, reaching 69.9% severity accuracy, 70.9% CWE accuracy, and an exact CVSS vector on 48.8% of test CVEs, +2.8 to +4.7 points ahead of TF-IDF logistic regression with paired bootstrap intervals that exclude zero. The two trained systems err on different CVEs, and one or the other is correct on 75.8% of severity labels. Computing the score from the predicted vector improves the linear baseline and makes every prediction internally consistent, at the cost of over-predicting the CRITICAL tier. We release code, configuration, and per-CVE predictions, and every number in this paper is regenerated from the pinned snapshot.

1 Introduction

A vulnerability disclosure tells a defender what is broken, not how urgently to fix it. Each disclosed flaw receives a CVE identifier and a short text description, and a team facing hundreds of new CVEs a week needs more than that to decide where to start. They rely on structured *enrichment*: a CVSS v3.1 vector that records how the flaw is exploited and what it affects, the base score (0–10) and severity tier that follow from the vector, and a CWE category for the type of weakness (FIRST, 2019). Vulnerability scanners, patch managers, and incident response playbooks all sort by these fields.

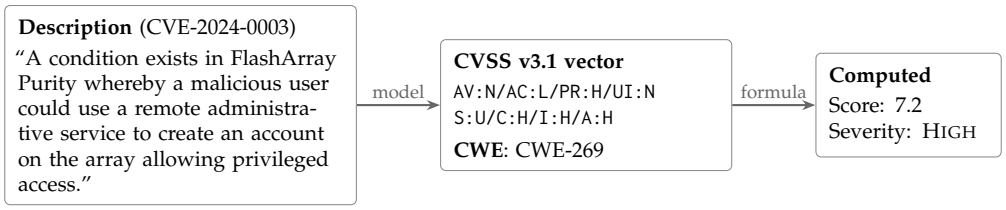


Figure 1: CVE enrichment for CVE-2024-0003. A model reads the description and predicts the eight CVSS components and the CWE. Under formula scoring, the score and severity are then computed from the predicted vector with the CVSS v3.1 equations, so the three always agree.

For most CVEs the enrichment came from the National Vulnerability Database (NVD) at NIST, and the NVD has not kept pace with disclosure volume. A backlog formed in early 2024. On 15 April 2026, NIST announced that it would enrich only CVEs in CISA's Known Exploited Vulnerabilities catalog, CVEs in software used by the federal government, and critical software as defined under Executive Order 14028, with every other CVE marked "Lowest Priority – not scheduled for immediate enrichment" (National Institute of Standards and Technology, 2026). For a typical new CVE, a defender now receives the description and whatever score the reporting organization chose to attach.

We formalize the resulting problem as a concrete prediction task we call **CVE enrichment**: given the description of a CVE, predict the eight CVSS v3.1 vector components, the base score, the severity tier, and the CWE. For example (Figure 1), CVE-2024-0003 describes a storage array in which "a malicious user could use a remote administrative service to create an account on the array allowing privileged access." A good enrichment model should read this sentence and recover that the attack arrives over the network, that it requires administrative privileges to begin with, that it compromises confidentiality, integrity, and availability, and that the weakness is improper privilege management, and it should then arrive at the same 7.2 that a NIST analyst assigned.

In this paper, we make progress on two fronts for CVE enrichment:

- **Evaluation.** How should enrichment models be evaluated so that reported accuracy reflects performance on the CVEs that have not been disclosed yet?
- **Modeling.** Which models recover the labels from text, and should the CVSS score be predicted or computed?

Sound evaluation is a prerequisite for progress on this task. Vulnerability descriptions are formulaic, and near-duplicate descriptions of the same product appear weeks or months apart, so a random split lets a model see the test distribution during training. Several public models are trained this way. We instead split by time: we train on CVEs from 2022 and 2023, taken from a pinned snapshot of the NVD feeds, and we test on every usable CVE from 2024. We also implement the CVSS v3.1 base-score equations in full and verify that they reproduce the NVD score for all 86,870 CVEs in the data, so that any error in a computed score can be attributed to the predicted vector.

With the benchmark in hand, we ask which models recover the labels and how the score should be produced. We compare TF-IDF classifiers, which prior work has found hard to beat on this kind of text (Talibzade et al., 2025), with a 3.8B-parameter instruction-tuned language model used zero-shot and with the same model fine-tuned with LoRA. Because the CVSS base score and severity are deterministic functions of the eight components, we evaluate every system under two schemes: predicting the score directly, or predicting the components and computing the score with the formula, as Elbaz et al. (2020) and Shahid

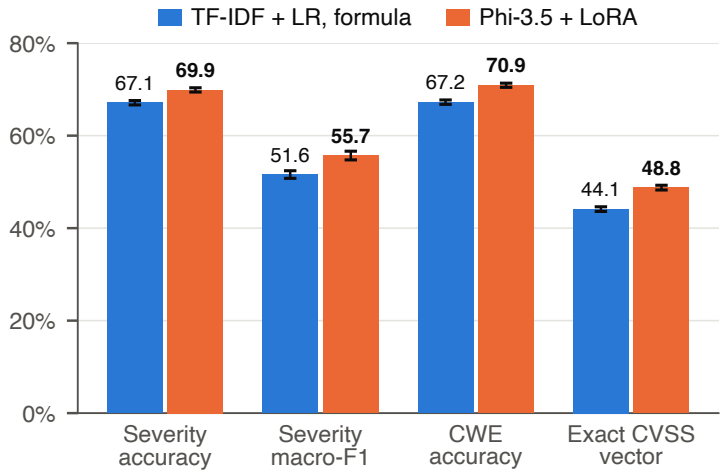


Figure 2: Severity accuracy, severity macro-F1, CWE accuracy, and exact-vector rate for fine-tuned Phi-3.5 and TF-IDF logistic regression with formula scoring on the full 2024 test set (36,325 CVEs). Error bars are 95% bootstrap intervals.

and Debar (2021) did. The second scheme guarantees that score, severity, and vector agree, and we measure what it adds over direct prediction and whether a language model's written score agrees with the vector it writes.

The fine-tuned model is the strongest system on every field (Figure 2). On the full 2024 test year it reaches 69.9% severity accuracy and 70.9% CWE accuracy and predicts the exact CVSS vector for 48.8% of CVEs, +2.8 to +4.7 points ahead of TF-IDF logistic regression. The margin is modest, and the structure of the errors is more informative than the margin. The two trained systems disagree on a large minority of CVEs, and one or the other is correct on 75.8% of severity labels. When the fine-tuned model misses the vector, it usually misses by one component, most often privileges required, on CVEs whose descriptions do not state who the attacker is. Both systems degrade on the shortest descriptions and on the LOW tier, which makes up under 2% of training data. After 2.5 hours of fine-tuning, the model's written score agrees with its own vector on 99.8% of CVEs, against 3.4% for the same model used zero-shot.

In sum, we contribute a leakage-free benchmark for CVE enrichment built on a pinned NVD snapshot and a verified CVSS calculator, a comparison of classical, zero-shot, and fine-tuned models on every enrichment field with paired confidence intervals, and an analysis of direct against formula-based scoring and of the self-consistency of language-model outputs. We release code, configuration, and per-CVE predictions for every system, and every number in this paper is regenerated from the snapshot.

2 Related Work

CVSS vector prediction. Elbaz et al. (2020) predicted each CVSS metric from a bag-of-words description with a linear model and derived the score from the predicted vector, with an emphasis on decisions that an analyst could inspect at disclosure time. CVSS-BERT (Shahid and Debar, 2021) trained one BERT classifier per CVSS v3.1 metric and computed the score with the equations of the specification. Marchiori et al. (2025) compared general-purpose LLMs with embedding-based classifiers on the same task and found the classifiers stronger on the three impact metrics. We keep the predict-the-vector design of these works and add a fine-tuned small language model, a strict temporal split, and a direct comparison against score regression.

Table 1: Data splits, with severity columns giving the share of CVEs (%) in each tier.

Split	CVEs	Critical	High	Medium	Low	CWEs
Train (2022–23)	40,436	15.4	39.3	43.7	1.7	464
Validation (2022–23)	10,109	15.4	39.3	43.7	1.7	293
Test (2024)	36,325	13.2	36.2	48.6	1.9	512

Severity and CWE classification. VLAI (Bonhomme and Dulaunoy, 2025) fine-tunes RoBERTa on more than 600,000 vulnerabilities to predict the severity tier. Its training set spans many years and includes 2024, so there is no fair way to score it on our test set (Section 3). For CWE assignment, Aota et al. (2020) used bag-of-words features with random forests, and Mosievskiy (2026) reports that a 125M-parameter RoBERTa competes with much larger LLMs.

LLMs for vulnerability triage. Talibzade et al. (2025) found that TF-IDF classifiers beat general-purpose LLMs with generic prompts on CWE and severity assignment and attributed this to the schematic wording of CVE descriptions. Our zero-shot results agree with theirs. CVE-LLM (Ghosh et al., 2025) fine-tunes an LLM with a domain ontology to evaluate medical-device vulnerabilities, and Torkamani et al. (2025) use LLMs to assign CWEs and severity inside a triage tool. On the related task of predicting exploitation, Roytman (2025) reports that frontier LLMs fall well short of the statistical EPSS model and cost far more to run at scale, and industry systems such as VulnWatch (Databricks, 2025) prioritize vulnerabilities with hosted LLMs. Our setting differs from these systems in two ways: everything runs on local hardware, and every output is scored on vulnerabilities disclosed after the training data.

3 Task and Data

Task. The input is the English description of a CVE. The outputs are the eight CVSS v3.1 base metrics (attack vector, attack complexity, privileges required, user interaction, scope, and the confidentiality, integrity, and availability impacts), the base score, the severity tier (CRITICAL, HIGH, MEDIUM, LOW), and the CWE identifier. Figure 1 shows one example.

Source. We take NVD records from the Fraunhofer FKIE mirror of the NVD JSON feeds (Fraunhofer FKIE, 2026), pinned to release `v2026.10.05-000016`. NVD revises records over time, and a fixed release keeps every number in this paper reproducible.

Filtering. We keep a CVE if it has an English description of at least 20 characters, a CVSS v3.x base metric with a severity tier, and a specific CWE, excluding the placeholders `NVD-CWE-noinfo` and `NVD-CWE-Other`. When a CVE carries several CVSS entries we use the primary score assigned by NIST. This leaves 86,870 CVEs from 2022 to 2024, of which 776 use CVSS v3.0 and the rest v3.1. The base-score equations of the two versions agree on every one of them.

Temporal split. CVEs with 2022 and 2023 identifiers form a pool of 50,545 CVEs, which we split 80/20 at random with severity stratification into 40,436 training and 10,109 validation CVEs. All 36,325 usable CVEs with 2024 identifiers form the test set. We use validation data only to choose hyperparameters and checkpoints. Table 1 gives the label distribution of each split.

Label distribution. LOW accounts for under 2% of every split. CWEs follow a long tail: the training set contains 464 distinct CWEs and the test set 512, and the 100 most frequent

training CWEs cover 91.7% of test CVEs.

CVSS calculator. We implement the CVSS v3.1 base-score equations (FIRST, 2019) in full, including the floating-point-safe round-up that the specification requires. Applied to the true vectors, the calculator reproduces the NVD base score and severity for all 86,870 CVEs, so any error in a formula-based score originates in the predicted vector.

Excluded baseline. VLAI (Bonhomme and Dulaunoy, 2025) is a public severity classifier whose training data, the `CIRCL/vulnerability-scores` dataset, uses a random split over all years. Rows we sampled from its training split include CVEs from 2024, 2025, and 2026. Scoring it on the 2024 test set would measure memorization as much as generalization, so we exclude it.

4 Methods

4.1 Direct and Formula Scoring

Every system predicts the eight vector components and the CWE. We then produce the score and severity in one of two ways. Under *direct* scoring, a separate model predicts the score by regression and the severity by classification, or a language model writes both itself. Under *formula* scoring, we apply the CVSS v3.1 equations to the predicted components, so that score, severity, and vector agree by construction. The TF-IDF systems share one ridge regression for the direct score, so their direct score errors are identical and we report them once.

4.2 TF-IDF Baselines

We represent each description with TF-IDF over unigrams and bigrams, with sublinear term frequency, a minimum document frequency of 2, and at most 50,000 features. We keep stop words, because "not" and "without" change the meaning of a description. We train one classifier per output with scikit-learn (Pedregosa et al., 2011). Logistic regression (LR) chooses its regularization strength $C \in \{0.3, 1, 3, 10\}$ by validation accuracy, separately for severity, for CWE, and for the components, with one C shared by all eight components. Class-balanced LR uses the same setup with class weights inversely proportional to class frequency. We also include an untuned random forest (RF) with 300 trees for severity and CWE and 100 trees per component. The direct score comes from ridge regression with $\alpha \in \{0.3, 1, 3, 10\}$ chosen by validation MAE. The CWE classifiers learn the 100 most frequent training CWEs plus an OTHER class, and we score them against the true CWE, so an OTHER prediction always counts as an error.

4.3 Phi-3.5-mini

Our language model is Phi-3.5-mini-instruct, a 3.8B-parameter decoder (Abdin et al., 2024) whose training data ends in October 2023, before the test period.

Fine-tuning. We fine-tune with LoRA (Hu et al., 2022) in MLX (Hannun et al., 2023) on a single Apple M5 Pro with 64 GB of unified memory. Adapters of rank 16 (scale 20, dropout 0.05) sit on the last 16 transformer layers and amount to 12.6M trainable parameters, or 0.329% of the model. We train for 5,000 steps at batch size 8, one pass over the training set, with a learning rate of 10^{-4}, 100 warm-up steps, and cosine decay to 10^{-5}, and the loss covers only the answer tokens. We drop the 561 training examples (1.4%) longer than 768 tokens to bound memory and keep every test CVE regardless of length. The target is a single JSON object that lists the eight components first and then the CWE, the score,

Table 2: Validation metrics of the fine-tuned model under formula scoring at each saved checkpoint, on the same 1,000 validation CVEs.

Step	Severity acc.	Macro-F1	CWE acc.	Score MAE	Exact vector
1,000	68.2	50.8	65.4	0.83	52.9
2,000	72.6	56.9	67.2	0.73	55.4
3,000	74.4	58.5	71.1	0.66	62.2
4,000	76.0	62.1	73.4	0.65	63.0
5,000	76.3	58.6	74.2	0.63	63.9

Table 3: All systems on the full 2024 test set (36,325 CVEs). Values are percentages except score MAE. ± gives the half-width of the 95% bootstrap interval. Bold marks the best value in each column. The TF-IDF direct systems share one ridge regression, so their score MAE is identical.

	Severity		CWE	Score	Vector	
System	Acc.	Macro-F1	Acc.	MAE ↓	Partial	Exact
TF-IDF + LR, direct	67.1 ±0.5	47.9 ±0.6	67.2 ±0.5	0.98 ±0.01	86.9 ±0.2	44.1 ±0.5
TF-IDF + LR, formula	67.1 ±0.5	51.6 ±0.8	67.2 ±0.5	0.94 ±0.01	86.9 ±0.2	44.1 ±0.5
TF-IDF + LR balanced, direct	65.7 ±0.5	51.9 ±0.8	67.2 ±0.5	0.98 ±0.01	86.0 ±0.2	40.5 ±0.5
TF-IDF + LR balanced, formula	67.1 ±0.5	53.9 ±0.9	67.2 ±0.5	0.98 ±0.01	86.0 ±0.2	40.5 ±0.5
TF-IDF + RF, direct	66.9 ±0.5	47.1 ±0.6	65.2 ±0.5	0.98 ±0.01	86.1 ±0.2	41.8 ±0.5
TF-IDF + RF, formula	64.0 ±0.5	47.8 ±0.6	65.2 ±0.5	1.03 ±0.01	86.1 ±0.2	41.8 ±0.5
Phi-3.5 + LoRA, direct	**69.9** ±0.5	**55.7** ±0.9	**70.9** ±0.5	**0.85** ±0.01	**87.9** ±0.2	**48.8** ±0.5
Phi-3.5 + LoRA, formula	**69.9** ±0.5	**55.7** ±0.9	**70.9** ±0.5	**0.85** ±0.01	**87.9** ±0.2	**48.8** ±0.5

and the severity, so the model commits to a vector before it writes a score. We evaluate checkpoints at steps 1,000 through 5,000 on the same 1,000 validation CVEs (Table 2) and keep the one with the best severity accuracy under formula scoring. Decoding is greedy.

Zero-shot. The same base model, without an adapter, receives the description and a prompt that lists every field and its allowed values (Appendix A). Zero-shot output is verbose and slow to generate for a whole year of CVEs, so we evaluate it on a random sample of 2,000 test CVEs and rescore every other system on the same sample.

4.4 Metrics

We evaluate severity with accuracy and with macro-F1 over the four tiers. Macro-F1 weights LOW as heavily as MEDIUM and so exposes failures on the rare tiers that accuracy hides. We evaluate CWE prediction with top-1 accuracy, the score with mean absolute error (MAE), and the vector with per-component accuracy, the mean fraction of components correct (*partial match*), and the fraction of CVEs with all eight components correct (*exact match*). A language-model output that is not valid JSON counts as wrong on every field and is excluded from the score MAE, and we report the parse rate separately. All 95% confidence intervals come from 1,000 bootstrap resamples of test CVEs.

5 Results

We synthesize the main findings from our evaluation, with results summarized in Table 3 and Table 4.

Table 4: Recall and F1 (%) by true severity tier on the 2024 test set.

	Critical		High		Medium		Low	
System	Rec.	F1	Rec.	F1	Rec.	F1	Rec.	F1
TF-IDF + LR, direct	50.7	51.4	62.2	61.1	77.9	77.1	1.1	2.2
TF-IDF + LR, formula	73.3	56.1	62.6	62.6	71.3	76.8	6.2	10.9
TF-IDF + LR bal., formula	57.1	52.8	61.9	62.2	75.6	77.2	19.2	23.2
Phi-3.5 + LoRA, formula	76.9	59.7	63.6	64.6	75.1	79.0	11.4	19.6

Fine-tuning a small language model outperforms TF-IDF on every field, by a modest margin. Table 3 reports every system on the 36,325 test CVEs. The fine-tuned Phi-3.5 model leads on every metric, reaching 69.9% severity accuracy, 70.9% CWE accuracy, a score MAE of 0.85, and an exact vector on 48.8% of CVEs. Because every system is scored on the same CVEs, we compare them with a paired bootstrap against TF-IDF logistic regression with formula scoring. Relative to that baseline, the fine-tuned model improves severity accuracy by +2.8 points (95% CI +2.4 to +3.1), macro-F1 by +4.1 (+3.0 to +5.2), CWE accuracy by +3.7 (+3.4 to +4.0), score MAE by -0.09 (-0.10 to -0.08), and the exact-vector rate by +4.7 (+4.3 to +5.0). No interval includes zero. The gains are consistent, but they are a few points, and TF-IDF remains a strong baseline that any method for this task should be required to beat.

Computing the score from the predicted vector helps the linear model and trades precision for recall on the Critical tier. For logistic regression, formula scoring lowers score MAE from 0.98 to 0.94 and raises macro-F1 from 47.9% to 51.6% while accuracy stays at 67.1%. The gain comes from the two rare tiers (Table 4): LOW F1 rises from 2.2% to 10.9%, and CRITICAL F1 from 51.4% to 56.1% as CRITICAL recall climbs from 50.7% to 73.3%. The improvement has a cost. The formula system predicts CRITICAL for 7,755 CVEs where NVD has 4,800, and its precision on that tier falls from 52.1% to 45.4%. It scores 17.7% of test CVEs at exactly 9.8, the score of the most common critical vector, against 10.5% in NVD, and its mean signed score error is +0.33 compared with +0.08 for ridge regression. For the random forest, whose component predictions are weaker, formula scoring passes those errors through to the score: accuracy falls from 66.9% to 64.0% and MAE rises from 0.98 to 1.03.

The fine-tuned model learns to apply the formula itself. The fine-tuned model shows almost no difference between the two scoring modes. The score it writes matches the CVSS formula applied to its own vector on 99.8% of test CVEs, and the severity it writes matches the tier of its own score on 100.0% (Table 6). Trained on targets that list the components before the score, the model behaves as if it computes the score from the vector it has just written.

Errors concentrate in the Low tier and in the privileges-required component. LOW is the hardest tier for every system (Table 4). It makes up under 2% of the training data, and both trained systems most often label a true LOW CVE as MEDIUM. Class-balanced weights raise LOW recall to 19.2% for TF-IDF, and the fine-tuned model reaches 11.4% without any rebalancing. Figure 3 shows that most of the remaining confusion falls between adjacent tiers. Across components (Figure 4), attack complexity, scope, attack vector, and user interaction exceed 90% for both systems, because descriptions usually state them outright. Privileges required is the hardest component, at 76.5% for TF-IDF and 78.8% for the fine-tuned model, followed by the three impact components. The fine-tuned model gains most on confidentiality (82.9% to 84.8%), integrity (82.9% to 84.5%), and privileges required, the components that depend least on surface keywords. Marchiori et al. (2025) found that general-purpose LLMs trail embedding-based classifiers on the impact components, and

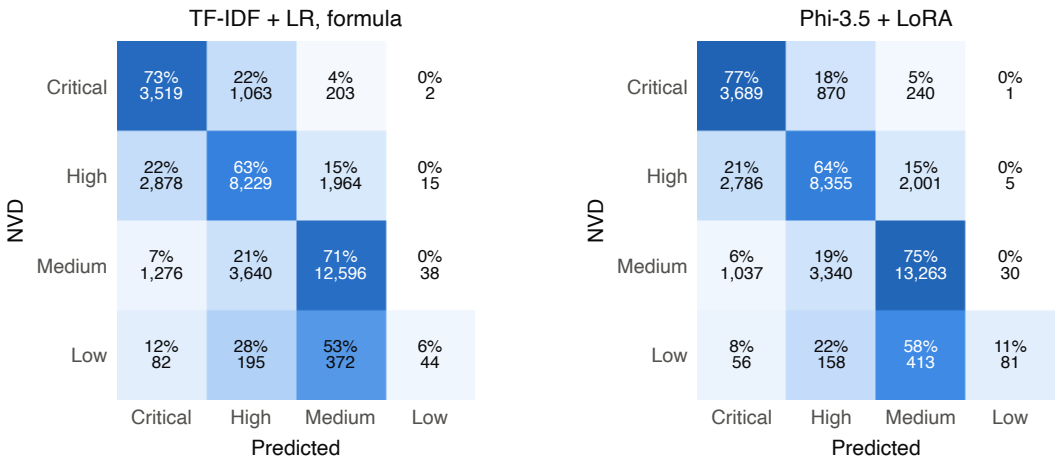


Figure 3: Severity confusion matrices on the 2024 test set, normalized by true tier, for TF-IDF logistic regression with formula scoring (left) and fine-tuned Phi-3.5 (right).

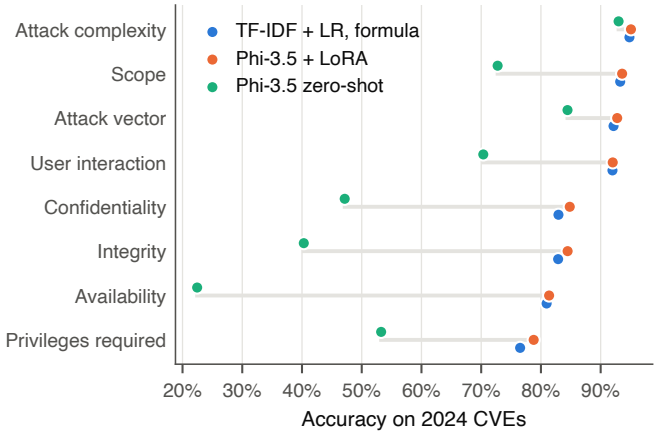


Figure 4: Accuracy per CVSS component on 2024 CVEs. TF-IDF and the fine-tuned model are scored on the full test set, the zero-shot model on its 2,000-CVE sample.

we find that after fine-tuning these are among the components where the language model gains most.

Only the generative model can reach rare CWEs. The TF-IDF classifiers can output only the 100 most frequent training CWEs and therefore score 0% on the 3,023 test CVEs whose CWE is rarer. The fine-tuned model writes the CWE as text and identifies 7.0% of them correctly (Figure 5). Common, explicitly named weaknesses such as cross-site scripting and SQL injection are close to solved by both systems.

Both trained systems over-predict the most common critical score. Ridge regression produces a smooth spread of scores that NVD never assigns and rarely predicts the extremes (Figure 6). The fine-tuned model's scores lie on the same discrete values as NVD's, but it predicts 9.8 for 20.0% of test CVEs where NVD assigns it to 10.5%. It predicts CRITICAL for 7,568 test CVEs against 4,800 in NVD, with 48.7% precision, and like formula-scored logistic regression it runs high on average, with a mean signed error of +0.33.

Zero-shot prompting fails on this task, and its outputs contradict themselves. Without fine-tuning, the same base model is far weaker (Table 5). It returned valid JSON for 99.7%

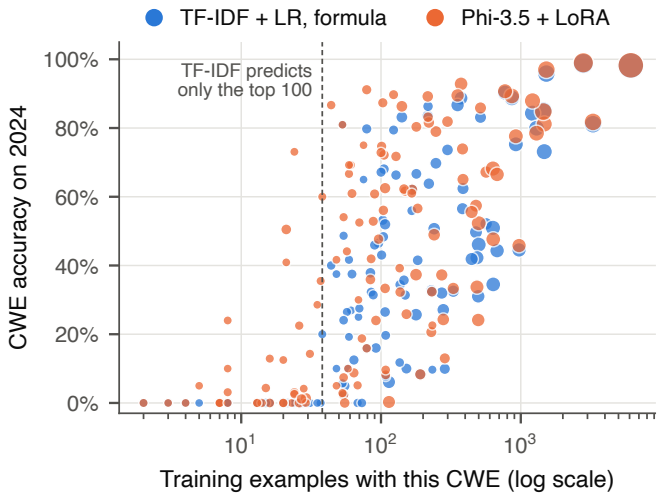


Figure 5: CWE accuracy on the 2024 test set against the number of training examples with each CWE, for every CWE with at least 20 test CVEs. Point size grows with test count.

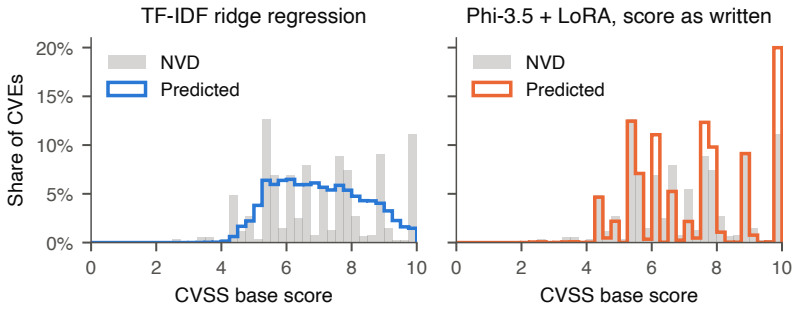


Figure 6: Distribution of predicted and NVD base scores on the 2024 test set in bins of 0.25, for ridge regression (left) and the fine-tuned model with scores as written (right).

Table 5: Comparison on the same random sample of 2,000 test CVEs, on which every system is scored.

	Severity		CWE	Score	Vector	
System	Acc.	Macro-F1	Acc.	MAE ↓	Partial	Exact
TF-IDF + LR, formula	65.3 ±2.1	47.6 ±1.8	66.2 ±2.2	0.99 ±0.06	86.5 ±0.7	42.2 ±2.2
Phi-3.5 zero-shot, direct	20.8 ±1.7	15.4 ±2.4	25.7 ±2.0	1.58 ±0.05	60.5 ±0.7	2.4 ±0.6
Phi-3.5 zero-shot, formula	43.5 ±2.3	26.8 ±1.7	25.7 ±2.0	2.08 ±0.08	60.5 ±0.7	2.4 ±0.6
Phi-3.5 + LoRA, formula	**69.6** ±2.0	**52.8** ±3.3	**68.8** ±2.1	**0.87** ±0.05	**87.5** ±0.7	**48.0** ±2.1

of the 2,000 sampled CVEs, and 11 of the parsed outputs used a component value outside the allowed set. Its written severity is biased upward: it labels 62.4% of the sample CRITICAL against 13.3% in NVD, predicts MEDIUM only 9 times although 47.5% of these CVEs are MEDIUM, and is correct on 20.8% of CVEs. The written score matches the formula for the written vector on 3.4% of CVEs, and the written severity matches the tier of the written score on 37.0% (Table 6). Computing severity from the vector removes the inconsistency and lifts severity accuracy to 43.5%, still far below the fine-tuned model (69.6%) and TF-IDF (65.3%) on the same CVEs, while score MAE worsens from 1.58 to 2.08 because the zero-shot vectors are wrong in ways that move the score a long way. The per-component accuracies explain this (Figure 4): attack complexity (93%) and attack vector (84%) approach the

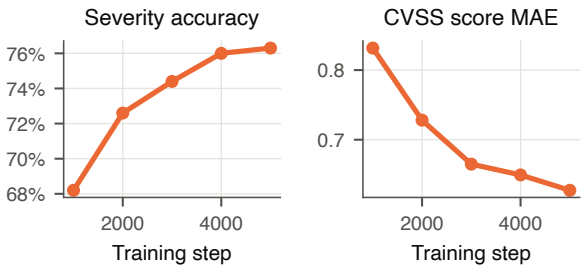


Figure 7: Validation severity accuracy and score MAE of the fine-tuned model with formula scoring at each saved checkpoint, on 1,000 validation CVEs.

trained systems, but availability impact is correct on only 22% of CVEs, integrity on 40%, and confidentiality on 47%. The zero-shot model reads how an attack happens reasonably well and guesses what it costs.

Table 6: Self-consistency of language-model outputs (%), where *Score = formula* is the share of outputs whose written score equals the CVSS v3.1 score of the written vector and *Severity = tier of score* is the share whose written severity matches the tier of the written score.

Model	Valid JSON	Score = formula	Severity = tier of score
Phi-3.5 zero-shot	99.7	3.4	37.0
Phi-3.5 + LoRA	100.0	99.8	100.0

Training is cheap, and validation accuracy had not plateaued. Validation severity accuracy rose at every checkpoint, and we selected the last one at step 5,000 (Table 2, Figure 7). Macro-F1 peaked earlier, at step 4,000 with 62.1%, so a deployment that weights the rare tiers more heavily might prefer that checkpoint. Table 7 gives wall-clock times. Fine-

Table 7: Wall-clock time on a single Apple M5 Pro (64 GB), with inference run in small batches of at most 32 CVEs and without optimization for speed.

Step	Time
All TF-IDF baselines, tuning and test (CPU)	9 min
LoRA fine-tuning, 5,000 steps	2.5 h
LoRA inference, 36,325 test CVEs	4.9 h

tuning took 2.5 hours and scoring the full test year 4.9 hours, against 9 minutes for all TF-IDF baselines together, including tuning.

6 Discussion

In our evaluation, a fine-tuned small language model recovers NVD enrichment from text more accurately than classical baselines, but by less than the difference in model size would suggest. We examine where the two trained systems disagree, what their errors look like on individual CVEs, and what this implies for deploying either system, and we outline directions for future work.

The two trained systems make complementary errors. A difference of a few points in accuracy says little about whether two systems make the same mistakes, and on this test set they do not. Both systems label the severity correctly on 61.3% of CVEs and both fail

Table 8: Test CVEs discussed in Section 6, with the NVD labels and the output of the fine-tuned model. The last column lists the vector components the model got wrong.

CVE	Description	NVD	Predicted	Wrong
CVE-2024-0003	A condition exists in FlashArray Purity whereby a malicious user could use a remote administrative service to create an account on the array allowing privileged access.	7.2 High, CWE-269	9.8 Critical, CWE-287	PR
CVE-2024-0008	Web sessions in the management interface in Palo Alto Networks PAN-OS software do not expire in certain situations, making it susceptible to unauthorized access.	8.8 High, CWE-613	9.8 Critical, CWE-613	UI
CVE-2024-0257	RoboDK v5.5.4 is vulnerable to heap-based buffer overflow while processing a specific project file. The resulting memory corruption may crash the application.	3.3 Low, CWE-122	7.8 High, CWE-787	C, I, A
CVE-2024-10214	Mattermost versions 9.11.X <= 9.11.1, 9.5.x <= 9.5.9 icorrectly issues two sessions when using desktop SSO - one in the browser and one in desktop with incorrect settings.	3.5 Low, CWE-303	9.8 Critical, CWE-287	PR, UI, C, I, A
CVE-2024-0204	Authentication bypass in Fortra's GoAnywhere MFT prior to 7.4.1 allows an unauthorized user to create an admin user via the administration portal.	9.8 Critical, CWE-425	9.8 Critical, CWE-287	none
CVE-2024-0149	NVIDIA GPU Display Driver for Linux contains a vulnerability which could allow an attacker unauthorized access to files. A successful exploit of this vulnerability might lead to limited information disclosure.	3.3 Low, CWE-125	3.3 Low, CWE-200	none

on 24.2%. On the remaining 8.6% only the fine-tuned model is correct, and on 5.9% only TF-IDF is. One or the other is therefore correct on 75.8% of CVEs, well above the 69.9% of the better system alone. The pattern repeats for the CWE (either correct on 73.8% against 70.9%) and for the exact vector (53.3% against 48.8%). We did not build an ensemble, so these figures are an upper bound rather than a result, but they indicate that the two systems read descriptions differently and that a combination is worth pursuing.

Most vector misses are single-component misses, and the missing information is often absent from the text. The exact-vector rate is a severe metric, since one wrong component out of eight counts as a miss, and most misses are small. The fine-tuned model gets every component right on 48.8% of test CVEs, exactly one wrong on 25.5%, two wrong on 12.7%, and three or more wrong on 13.0%, and TF-IDF has the same shape (44.1%, 28.1%, 14.1%, and 13.6%). When exactly one component is wrong, it is privileges required 38% of the time for the fine-tuned model and 38% for TF-IDF, with availability impact a distant second at 17% and 16%.

Table 8 shows what this looks like on individual CVEs. CVE-2024-0003, our running example, is a privileges-required miss. The description says a "malicious user" can create an account through "a remote administrative service." NIST read "administrative" as meaning the attacker already holds high privileges and scored the CVE 7.2, whereas the model read "malicious user" as an outsider, set privileges required to none, and produced 9.8 with every other component matching. A single ambiguous word turns a HIGH into a CRITICAL, which is why this one component accounts for so much of the gap between partial-match and exact-match accuracy. CVE-2024-0008 is a user-interaction miss of the same kind: PAN-OS management sessions that do not expire are scored 8.8 by NVD with user interaction required, and the model, seeing no mention of a user, scored them 9.8. In both cases the model's reading is defensible from the text, and the information it lacks is

Table 9: Mean absolute score error by true severity tier on the 2024 test set.

System	Critical	High	Medium	Low
TF-IDF ridge regression (direct)	1.23	0.80	0.96	2.91
TF-IDF + LR, formula	0.59	0.81	1.04	3.26
Phi-3.5 + LoRA, formula	0.53	0.76	0.92	2.90

Table 10: Severity accuracy and exact-vector rate (%) by quartile of description length on the 2024 test set. Quartile boundaries are 194, 285, and 444 characters.

	Shortest quartile		Second		Third		Longest quartile	
System	Sev.	Vector	Sev.	Vector	Sev.	Vector	Sev.	Vector
TF-IDF + LR, formula	59.7	38.8	67.2	38.4	71.0	50.7	70.6	48.7
Phi-3.5 + LoRA, formula	61.7	42.0	69.7	42.0	74.4	55.7	73.9	55.5

not in the description.

The Low tier fails for a reason beyond its rarity. Table 9 gives score MAE by true tier. The fine-tuned model's score is off by 0.53 on average for CRITICAL CVEs, by 0.76 for HIGH, by 0.92 for MEDIUM, and by 2.90 for LOW, and ridge regression, which never commits to a vector, does no better on LOW at 2.91. A LOW CVE that the model gets wrong is usually wrong by more than one tier.

The examples show why. CVE-2024-0257 describes a heap-based buffer overflow in RoboDK that "may crash the application." NVD scored it 3.3 with limited impact, because the attack needs a crafted project file and the worst outcome mentioned is a crash, while the model scored it 7.8 with high impact on all three dimensions, the typical score for a heap overflow in the training data. It applied the prior for the weakness type and ignored the limiting clause. CVE-2024-10214, a Mattermost bug that issues two sessions during desktop single sign-on, is scored 3.5 by NVD and 9.8 by the model, which read a session-handling flaw as an authentication bypass. Both errors run in the same direction: when a description contains words associated with serious weaknesses, the model scores the weakness rather than the sentence. Class-balanced weights help TF-IDF on this tier, and we did not try the equivalent for the fine-tuned model, which would be oversampling LOW during training.

Both systems degrade on short descriptions, and fine-tuning does not rescue them. Table 10 splits the test set into quartiles of description length. Both systems do worst on the shortest quarter, under 194 characters, where the fine-tuned model reaches 61.7% severity accuracy and TF-IDF 59.7%. Accuracy rises through the middle quartiles and levels off, with the fine-tuned model at 74.4% on the third quartile and 73.9% on the longest, and the exact-vector rate follows the same curve. The shortest descriptions are often a product name and a weakness class with nothing else, as in CVE-2024-0057 (".NET, .NET Framework, and Visual Studio Security Feature Bypass Vulnerability"), and for those the score must come from the prior for that vendor and weakness. The fine-tuned model's lead over TF-IDF is similar in every quartile, so the additional capacity does not specifically help short inputs.

Some CWE errors are disagreements with the analyst, not with the text. Two of the examples in Table 8 have an exact vector and a wrong CWE. For CVE-2024-0204, an authentication bypass in GoAnywhere MFT, NVD assigned CWE-425 (direct request) and the model CWE-287 (improper authentication). For CVE-2024-0149, an NVIDIA driver flaw that allows "unauthorized access to files" leading to "limited information disclosure," NVD as-

signed CWE-125 (out-of-bounds read) and the model CWE-200 (exposure of sensitive information), although the description never mentions a bounds violation. The NVD label records what the analyst learned from the advisory, and the model's label records what the description says. Some share of the CWE errors in Table 3 are of this kind, and the CWE accuracy of any description-only system is bounded by it.

Deployment. The results point to a simple division of labor. TF-IDF logistic regression trains in minutes, needs no GPU, and gives up a few points on every field, and it should be used with formula scoring, which costs nothing and repairs the rare tiers, unless precision on CRITICAL matters more than recall. The fine-tuned model is better on every field, produces consistent scores and vectors, and can name CWEs outside the frequent set, at a cost of 2.5 hours of training and about half a second per CVE at inference, which is fast enough for the daily volume of new disclosures. The zero-shot model should not be used for this task. Whichever system is deployed, the LOW tier and the privileges-required component should be treated as unreliable, and a CVE scored 9.8 by either trained system deserves review before it moves to the front of a queue.

Future work. Several extensions follow directly from these findings. Oversampling LOW or weighting its loss should be tried for the fine-tuned model, since the same change helped TF-IDF. A combination of the two trained systems, by agreement or by confidence, would test whether the 75.8% upper bound can be approached. A second training epoch is cheap, and the validation curve suggests it would help. Larger open models fine-tuned the same way, and an evaluation on CVSS v4.0 vectors, would show whether these results hold beyond the setting we study.

7 Limitations

Label noise. We treat NVD vectors as ground truth even though analysts disagree, NVD revises its records, and reporting organizations sometimes score the same CVE differently. Section 6 shows several errors that a reviewer would accept as alternative readings. We pin the data snapshot so that our numbers stay fixed as the labels change.

Single split and seed. We evaluate on one test year and train each model once, so the bootstrap intervals capture the sampling of test CVEs and nothing about variation from training. We fine-tune one 3.8B model for one epoch, and its validation accuracy was still rising at the end (Table 2), so larger models or longer training may do better. We did not run large hosted models, and we trained the fine-tuned model without rebalancing the rare tier.

No ensemble. We report the disagreement between the two trained systems as an upper bound on what a combination could achieve, and we did not build or test one.

CVSS version. New CVEs increasingly carry CVSS v4.0 vectors. The formula approach carries over, since v4.0 also computes the score from the vector, but we tested nothing on v4.0 data.

Zero-shot sample. We evaluate the zero-shot model on 2,000 random test CVEs rather than the full year, and the intervals in Table 5 reflect the smaller sample.

Input length. We use only English descriptions and cut them at 2,000 characters, which affects 1.4% of test CVEs.

8 Conclusion

We introduced CVE enrichment, the task of recovering the CVSS vector, score, severity tier, and CWE of a vulnerability from its text description, along with a leakage-free temporal benchmark built on a pinned NVD snapshot and a verified CVSS calculator. On a full year of held-out CVEs, a 3.8B-parameter model fine-tuned for 2.5 hours outperforms strong TF-IDF baselines on every field, and computing the score from the predicted vector, which costs nothing, is the single change that helps the classical baseline most. The two trained systems err on different CVEs, and either one is correct on 75.8% of severity labels, which we consider the most promising lead in these results. Where the fine-tuned model fails, it fails in ways that follow from the text: an ambiguous word about privileges, a weakness class whose typical score overrides a limiting clause, or a description too short to carry the information at all. Altogether, we argue that recovering most of the enrichment the NVD no longer provides is tractable with public data and modest compute, and that the remaining errors are as much a property of the descriptions as of the models.

References

Marah Abdin et al. Phi-3 technical report: A highly capable language model locally on your phone, 2024.

Masaki Aota, Hideaki Kanehara, Masaki Kubo, Noboru Murata, Bo Sun, and Takeshi Takahashi. Automation of vulnerability classification from its description using machine learning. In *IEEE Symposium on Computers and Communications (ISCC)*, 2020.

Cédric Bonhomme and Alexandre Dulaunoy. VLAI: A RoBERTa-based model for automated vulnerability severity classification, 2025.

Databricks. VulnWatch: AI-enhanced prioritization of vulnerabilities. https://www.databricks.com/blog/vulnwatch-ai-enhanced-prioritization-vulnerabilities, 2025.

Clément Elbaz, Louis Rilling, and Christine Morin. Fighting n-day vulnerabilities with automated CVSS vector prediction at disclosure. In *Proceedings of the 15th International Conference on Availability, Reliability and Security (ARES)*, 2020. doi: 10.1145/3407023.3407038.

FIRST. Common vulnerability scoring system v3.1: Specification document. https://www.first.org/cvss/v3.1/specification-document, 2019.

Fraunhofer FKIE. NVD JSON data feeds, release v2026.10.05-000016. https://github.com/fkie-cad/nvd-json-data-feeds, 2026.

Rikhiya Ghosh, Hans-Martin von Stockhausen, Martin Schmitt, George Marica Vasile, Sanjeev Kumar Karn, and Oladimeji Farri. CVE-LLM: Ontology-assisted automatic vulnerability evaluation using large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 2025. doi: 10.1609/aaai.v39i28.35139.

Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on apple silicon. https://github.com/ml-explore/mlx, 2023.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

Francesco Marchiori, Denis Donadel, and Mauro Conti. Can LLMs classify CVEs? investigating LLMs capabilities in computing CVSS vectors. In *TrustAICyberSec Workshop*, 2025.

Nikita Mosievskiy. Fine-tuning RoBERTa for CVE-to-CWE classification: A 125M parameter model competitive with LLMs, 2026.

National Institute of Standards and Technology. NIST updates NVD operations to address record CVE growth. https://www.nist.gov/news-events/news/2026/04/nist-updates-nvd-operations-address-record-cve-growth, April 2026. Accessed 2026-10-05.

Fabian Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Michael Roytman. Benchmarking LLMs on the vulnerability prioritization task. Empirical Security. https://research.empiricalsecurity.com/research/benchmarking-llms, 2025.

Mustafizur R. Shahid and Hervé Debar. CVSS-BERT: Explainable natural language processing to determine the severity of a computer security vulnerability from its description. In *20th IEEE International Conference on Machine Learning and Applications (ICMLA)*, 2021.

Rustam Talibzade, Idilio Drago, and Francesco Bergadano. On using LLMs for vulnerability classification. In *Proceedings of the 2025 Workshop on Large AI Systems and Models with Privacy and Security Analysis (LAMPS)*, 2025. doi: 10.1145/3733800.3763267.

Mohammad Jalili Torkamani, Joey NG, Nikita Mehrotra, Mahinthan Chandramohan, Padmanabhan Krishnan, and Rahul Purandare. Streamlining security vulnerability triage with large language models, 2025.

A Prompts

Both prompts are sent as a single user turn through the Phi-3.5 chat template, and descriptions longer than 2,000 characters (1.4% of test CVEs) are cut at that length.

Fine-tuned model (user turn).

```
You are a cybersecurity expert. Given the CVE description below, predict the
NVD enrichment fields.
Return ONLY a JSON object with no extra text.

CVE Description:
<CVE description>
```

Zero-shot model (user turn).

```
You are a cybersecurity expert specializing in vulnerability analysis. Given a
CVE description, predict the NVD CVSS v3.1 enrichment fields.

CVE Description:
<CVE description>

Respond ONLY with one JSON object with exactly these keys and allowed values:
{"attack_vector": "NETWORK"|"ADJACENT_NETWORK"|"LOCAL"|"PHYSICAL",
"attack_complexity": "LOW"|"HIGH", "privileges_required": "NONE"|"LOW"|"HIGH",
"user_interaction": "NONE"|"REQUIRED", "scope": "UNCHANGED"|"CHANGED",
"confidentiality": "NONE"|"LOW"|"HIGH", "integrity": "NONE"|"LOW"|"HIGH",
"availability": "NONE"|"LOW"|"HIGH", "cwe": "CWE-<number>", "cvss_score":
<number 0.0-10.0>, "severity": "CRITICAL"|"HIGH"|"MEDIUM"|"LOW"}
```

B Reproducibility

All code, configuration files, and per-CVE predictions are in the project repository, the NVD snapshot is pinned to FKIE release `v2026.10.05-000016`, and each step below is a single command that regenerates its part of the paper.

- Data preparation with `download_data.py` and `prepare_splits.py`.
- TF-IDF baselines with `baselines_classical.py`.
- LoRA training with `mlx_lm.lora` and `scripts/lora_phi35.yaml`.
- Checkpoint selection with `select_checkpoint.py`.
- Evaluation with `evaluate_lora.py`.
- Tables, figures, and numbers in the text with `make_tables.py` and `plot_results.py`.